PHYSIOCARE CLINIC PORTAL · V0.1

Technical Documentation

Architecture, data model, integrations, deployment pipeline, and implementation reference for developers and IT teams.

DOCUMENT TYPE

Technical Reference

AUDIENCE

Developers, DevOps, IT administrators

STACK

React · Vite · Supabase · Cloudflare R2

VERSION

June 2026

Table of Contents

1. System Architecture

PhysioCare is a client-side Single Page Application (SPA) with no dedicated backend server. Business logic runs in the browser; data persists to Supabase PostgreSQL or browser localStorage; files store in Cloudflare R2 via an edge Worker.

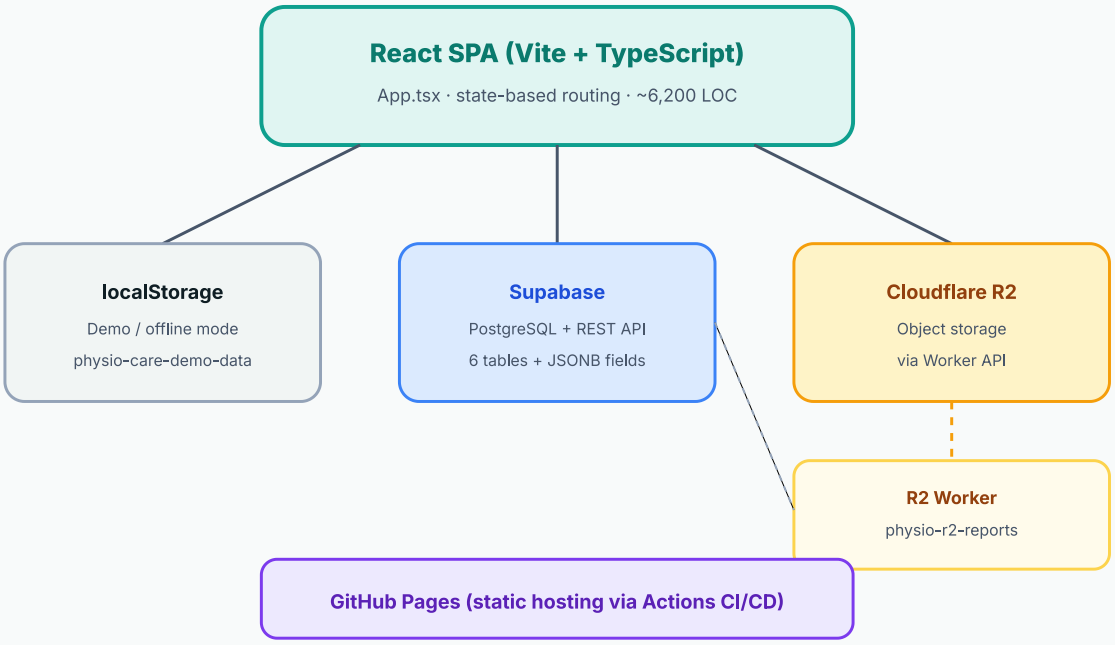

Figure 1 — High-level system architecture

Design Principles

- **No backend server:** Frontend calls Supabase and R2 Worker directly using public keys/tokens
- **Dual persistence:** Automatic fallback to localStorage when Supabase env vars are absent
- **Single-file UI:** All views live in `src/App.tsx` with internal page state (no React Router)
- **Build-time config:** All `VITE_*` variables embedded at compile time by Vite

2. Technology Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE
UI Framework	React	18.3	Component rendering
Language	TypeScript	5.6	Type safety across app
Build tool	Vite	5.4	Dev server, production bundling
Icons	lucide-react	0.468	Consistent iconography
Database	Supabase (PostgreSQL)	JS v2.45	Remote persistence
File storage	Cloudflare R2	—	Patient report files
Edge compute	Cloudflare Workers	Wrangler	R2 upload/download API
Hosting	GitHub Pages	Actions v4	Static site deployment

3. Project Structure

```
physio-clinic-portal/
├── src/
│   ├── App.tsx              # Main app (~6,200 lines): routing, views, handlers
│   ├── types.ts             # TypeScript interfaces & enums
│   ├── styles.css           # Global CSS design system
│   ├── main.tsx             # React entry point
│   ├── lib/
│   │   ├── supabase.ts      # DB client, mappers, CRUD helpers
│   │   └── r2.ts            # R2 upload/download client
│   └── data/
│       └── mockData.ts      # Seed data for demo mode
├── workers/r2-reports/
│   ├── src/index.ts         # Cloudflare Worker (upload/download/delete)
│   └── wrangler.toml        # Worker config, R2 binding, CORS origins
├── supabase_migration.sql   # Production DB schema + RLS + seed admin
├── .github/workflows/
│   └── deploy.yml           # GitHub Pages CI/CD
├── docs/                    # Business & technical documentation (this)
└── vite.config.ts           # Base path for GitHub Pages
```

4. Data Model

Central application state shape: `AppData` containing six entity collections.

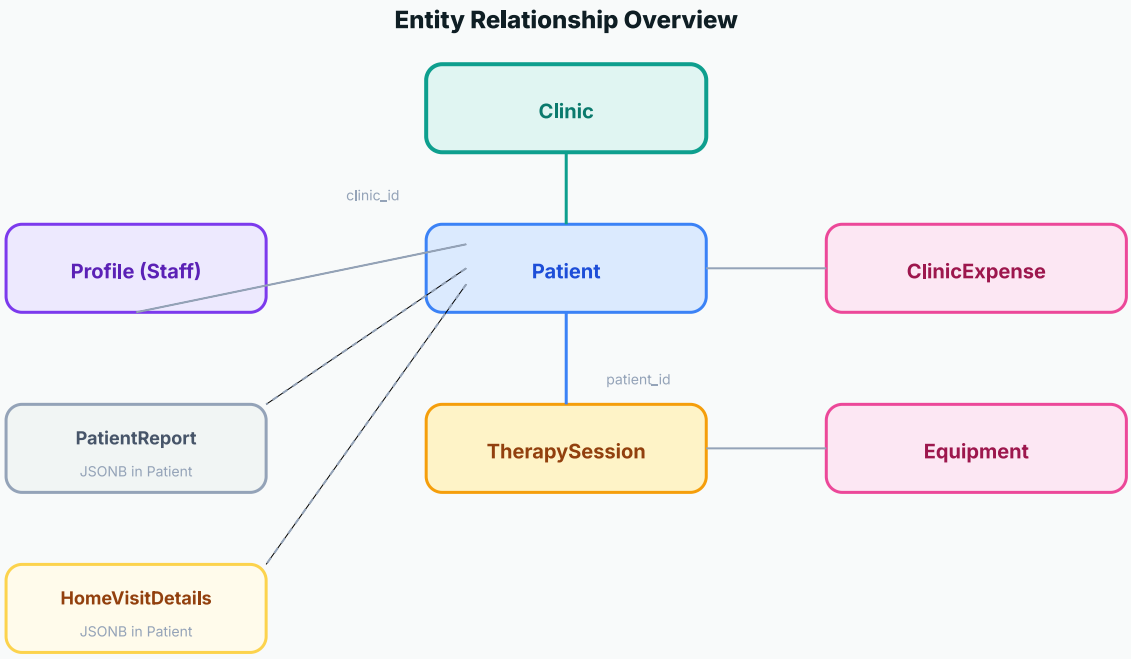

Figure 2 — Core entities and relationships

Key Types (src/types.ts)

ENTITY	NOTABLE FIELDS
`Patient`	`clinicId` (nullable), `homeVisitDetails?`, `reports[]`, clinical text fields
`TherapySession`	`sessionType` clinic|home, `therapyLevel`, `amountCollected`, `status`
`Profile`	`role` admin|staff, `status` pending|active|inactive, `clinicId`
`ClinicExpense`	`category`, `recurrence`, `amount`, `date`
`Equipment`	`category`, `condition`, `purchaseCost`

5. Application Routing & State

Navigation uses a `Page` union type and `useState<Page>` — no URL router. Sub-views use additional state:

PAGE KEY	VIEW	ACCESS
`dashboard`	Clinic Dashboard	All users
`homeDashboard`	Home Visit Dashboard	All users
`patients` / `patientEntry` / `patientDetail`	Patient list, add, profile	All users
`sessions` / `scheduleNew`	Session list, bulk scheduler	All users
`homeVisits`	Home visit management	All users
`calendar`	Clinic calendar	All users
`clinics` / `staff` / `expenses`	Admin modules	Admin only

Data Scoping

```
Admin  → visibleClinicIds = all clinic IDs
Staff  → visibleClinicIds = [currentUser.clinicId]
Patients/sessions with clinicId === null (home-only) → visible to all scoped users
```

6. Persistence Layer

Supabase Mode (production)

- Triggered when `VITE_SUPABASE_URL` + `VITE_SUPABASE_ANON_KEY` are set
- `loadRemoteData()` fetches all 6 tables in parallel on mount
- Every mutation calls Supabase REST API then `refreshRemoteData()`
- Row mappers convert snake_case DB columns ↔ camelCase TypeScript
- Expense/equipment mutations verify row count (prevent silent 0-row updates)

Demo Mode (localStorage)

- Key: `physio-care-demo-data`
- Seed from `mockData.ts` on first load
- Auth via `demoPasswords` map (no network)

7. Cloudflare R2 Worker API

Worker name: `physio-r2-reports` · Bucket: `physio-reports`

ENDPOINT	METHOD	DESCRIPTION
`/upload`	POST	Multipart: patientId, reportId, file → stores at `patients/{id}/reports/{reportId}/{filename}`
`/download?key=...`	GET	Streams file with correct Content-Type
`/patient-reports?patientId=...`	DELETE	Deletes all report objects for a patient

Security

- All endpoints require `Authorization: Bearer <R2_API_TOKEN>`
- CORS restricted to `ALLOWED_ORIGINS` in wrangler.toml (origin normalized, no path)
- Upload: 10 MB max, MIME type whitelist, filename sanitization
- Path traversal blocked on key and patientId parameters

Frontend Integration (src/lib/r2.ts)

- `isR2Configured` = both `VITE_R2_API_URL` and `VITE_R2_API_TOKEN` present at build time
- Stored keys prefixed with `patients/` → detected by `isStoredReportKey()`
- Download opens blob URL in new tab (avoids exposing token in URL for direct links)

8. Authentication & Authorization

ASPECT	IMPLEMENTATION
Auth mechanism	Custom profile login (NOT Supabase Auth JWT). Queries `profiles` table by email + password.
Session persistence	`localStorage` key `physio_session_user_id`
Inactivity timeout	30 minutes — auto logout on mouse/keyboard/scroll idle
Role enforcement	Frontend: adminOnly nav items, clinic-scoped data filter. DB: permissive RLS in migration SQL.
Staff signup	Creates profile with `status: pending`; admin sets `active`

> **Security note**
>
> Passwords are stored as plain text in the profiles table (documented in migration SQL). Production deployments should migrate to hashed passwords and stricter RLS policies aligned with the frontend role model.

9. Deployment

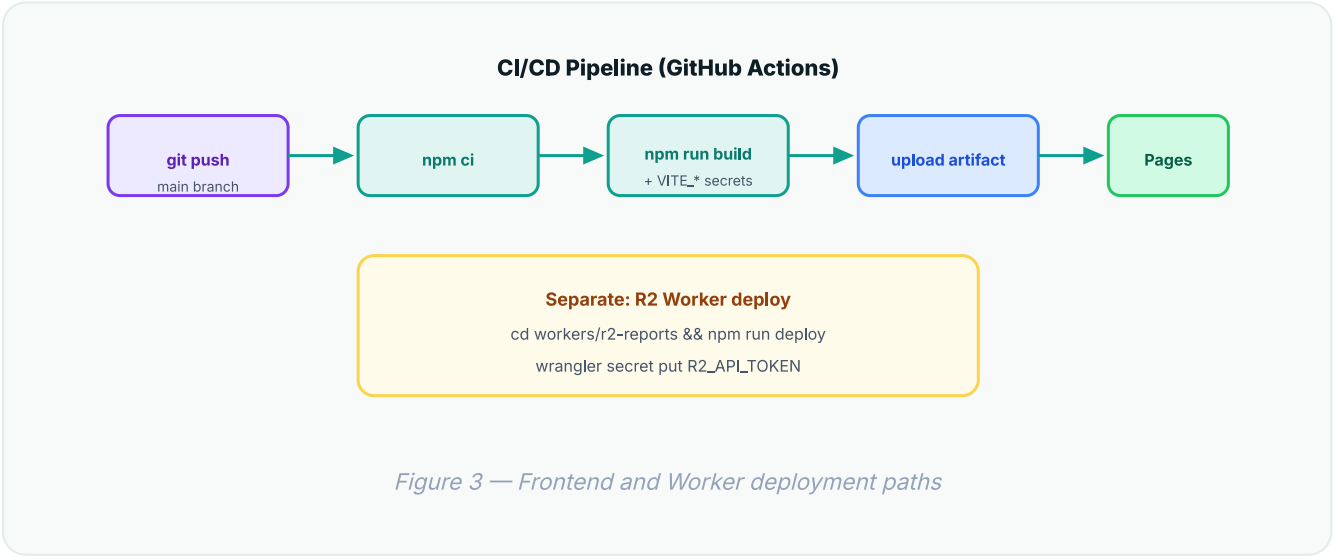

Figure 3 — Frontend and Worker deployment paths

10. Environment Variables

All frontend variables must be prefixed with `VITE_` and set at **build time**.

VARIABLE	REQUIRED	WHERE SET	DESCRIPTION
`VITE_SUPABASE_URL`	Production	GitHub Secret / .env.local	Supabase project URL
`VITE_SUPABASE_ANON_KEY`	Production	GitHub Secret / .env.local	Supabase anonymous API key
`VITE_R2_API_URL`	For file uploads	GitHub Secret / .env.local	Deployed Worker URL
`VITE_R2_API_TOKEN`	For file uploads	GitHub Secret / .env.local	Must match Worker secret
`VITE_BASE_PATH`	GitHub Pages	Auto in deploy.yml	`/{repo-name}/`

Worker secrets (Wrangler)

SECRET / VAR	DESCRIPTION
`R2_API_TOKEN`	Bearer token for API auth (wrangler secret)
`ALLOWED_ORIGINS`	CORS origins in wrangler.toml (host only for GitHub Pages)
`REPORTS_BUCKET`	R2 bucket binding (physio-reports)

11. Database Schema

Run `supabase_migration.sql` in Supabase SQL Editor. Tables:

TABLE	KEY COLUMNS
`clinics`	id, name, address, phone, active
`profiles`	id, email, password, role, clinic_id, status
`patients`	id, clinic_id (nullable), demographics, reports JSONB, home_visit_details JSONB
`therapy_sessions`	id, patient_id, clinic_id (nullable), scheduled_at, session_type, therapy_level, amount_collected, status
`clinic_expenses`	id, clinic_id, category, amount, date, recurrence, notes
`equipment`	id, clinic_id, name, category, condition, purchase_cost, serial_number

RLS: permissive `allow_all_*` policies. Grants on expenses/equipment for anon role.

12. Key Implementation Patterns

Atomic Home Visit Sync

`syncHomeVisitLog()` saves patient homeVisitDetails and creates/updates the linked TherapySession in one operation — avoids race conditions from dual Supabase calls.

Unified Modal Design System

All modals use: `modal-accent` → `modal-header` → `modal-body` → `modal-footer` CSS classes.

Patient Type Detection

```
function isHomeOnlyPatient(p) {
  return p.clinicId === null || Boolean(p.homeVisitDetails);
}
```

Multi-Patient Calendar Booking

Calendar slot booking supports selecting multiple patients; creates one TherapySession per patient at the same scheduledAt timestamp.

Local Development

```
# Frontend
cp .env.example .env.local   # fill in values
npm run dev                   # http://localhost:5173

# R2 Worker (separate terminal)
cd workers/r2-reports
npm run dev                   # http://localhost:8787
# Set VITE_R2_API_URL=http://localhost:8787 in .env.local
```

Default Admin Account (after migration)

Email: `admin@physiocare.local` · Password: `admin123` — change immediately in production.